

Foreign Data Wrappers (FDW) in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [FDW Architecture](#)
 3. [postgres_fdw](#)
 4. [file_fdw](#)
 5. [Other Common FDWs](#)
 6. [Cross-Database Queries](#)
 7. [Performance Optimization for FDW](#)
 8. [Security Considerations](#)
 9. [Production Monitoring Queries](#)
 10. [Common Mistakes](#)
 11. [Best Practices](#)
 12. [Interview Questions & Answers](#)
 13. [Exercises and Solutions](#)
 14. [Cross-References](#)
-

Learning Objectives

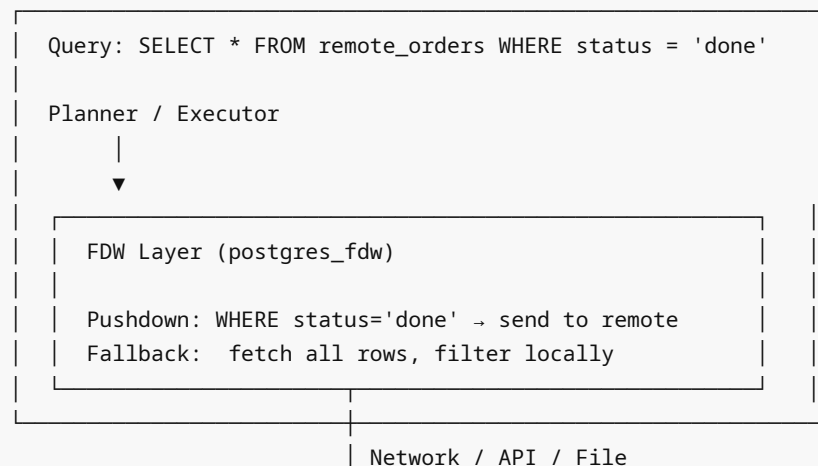
By the end of this module you will be able to:

- Explain the SQL/MED standard and how FDW implements it
 - Set up `postgres_fdw` to query remote PostgreSQL databases
 - Use `file_fdw` to query CSV/log files as SQL tables
 - Understand pushdown optimization and when it applies
 - Design cross-database ETL pipelines using FDW
 - Secure FDW connections with proper credential management
-

FDW Architecture

Foreign Data Wrapper Architecture (SQL/MED standard):

Local PostgreSQL Server:



Remote Data Source:

Remote PostgreSQL / MySQL / CSV file / S3 / REST API

SQL/MED components

Component	Description
Foreign Data Wrapper	The driver/plugin (e.g., postgres_fdw)
Foreign Server	A named connection to a remote data source
User Mapping	Maps local user to remote credentials
Foreign Table	Local table definition pointing to remote data

postgres_fdw

Setup: Querying a remote PostgreSQL database

```
-- === LOCAL SERVER (where queries run) ===

-- Step 1: Install the extension
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

-- Step 2: Create foreign server
CREATE SERVER remote_orders_server
    FOREIGN DATA WRAPPER postgres_fdw
    OPTIONS (
        host '10.0.1.50',
        port '5432',
        dbname 'orders_db',
        sslmode 'require',
        connect_timeout '10',
        application_name 'fdw_local_server'
    );

-- Step 3: Create user mapping (map local user → remote credentials)
CREATE USER MAPPING FOR CURRENT_USER
    SERVER remote_orders_server
    OPTIONS (
        user 'fdw_readonly_user',
        password 'secure_password'
    );

-- Also map other local users who need access
CREATE USER MAPPING FOR analytics_user
    SERVER remote_orders_server
    OPTIONS (user 'fdw_readonly_user', password 'secure_password');
```

```

-- Step 4a: Import entire schema (auto-discover tables)
CREATE SCHEMA remote_orders;
IMPORT FOREIGN SCHEMA public
    FROM SERVER remote_orders_server
    INTO remote_orders;

-- Step 4b: Or create individual foreign tables manually
CREATE FOREIGN TABLE orders_remote (
    id          BIGINT      OPTIONS (column_name 'id'),
    customer_id INT,
    status      TEXT,
    total       NUMERIC(12,2),
    created_at  TIMESTAMPTZ
)
SERVER remote_orders_server
OPTIONS (
    schema_name 'public',
    table_name  'orders',
    fetch_size  '10000' -- rows per network batch
);

-- Step 5: Query the foreign table
SELECT * FROM orders_remote WHERE status = 'completed' LIMIT 10;

-- Cross-server JOIN (local + remote)
SELECT
    c.name,
    SUM(r.total) AS remote_spend
FROM customers c -- local table
JOIN orders_remote r ON r.customer_id = c.id -- remote table
WHERE r.created_at > now() - interval '30 days'
GROUP BY c.name
ORDER BY remote_spend DESC
LIMIT 10;

```

IMPORT FOREIGN SCHEMA

```

-- Import all tables from remote schema
IMPORT FOREIGN SCHEMA public
    FROM SERVER remote_orders_server
    INTO remote_orders;

-- Import only specific tables
IMPORT FOREIGN SCHEMA public
    LIMIT TO (orders, order_items, products)
    FROM SERVER remote_orders_server
    INTO remote_orders;

-- Import all EXCEPT certain tables
IMPORT FOREIGN SCHEMA public

```

```

    EXCEPT (temp_table, staging_data)
    FROM SERVER remote_orders_server
    INTO remote_orders;

-- Check imported foreign tables
SELECT foreign_table_schema, foreign_table_name, foreign_server_name
FROM information_schema.foreign_tables;

```

Pushdown verification

```

-- Check if WHERE clause is pushed down to remote server
EXPLAIN VERBOSE
SELECT * FROM orders_remote WHERE status = 'completed' AND total > 100;
/*
  Foreign Scan on orders_remote
  Output: id, customer_id, status, total, created_at
  Remote SQL: SELECT id, customer_id, status, total, created_at
               FROM public.orders
               WHERE ((status = 'completed') AND (total > 100::numeric))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
               Pushed down! Remote server executes the filter.

*/

-- Check if aggregate is pushed down (postgres_fdw 9.6+)
EXPLAIN VERBOSE
SELECT customer_id, SUM(total) FROM orders_remote GROUP BY customer_id;
-- Look for "Remote SQL: SELECT customer_id, sum(total) GROUP BY 1"

-- Force local execution (no pushdown) to compare
SET enable_mergejoin = off;
SET enable_hashjoin = off;

```

file_fdw

`file_fdw` exposes files on the PostgreSQL server's filesystem as readable SQL tables.

```

CREATE EXTENSION IF NOT EXISTS file_fdw;

-- Create foreign server (just a placeholder for file_fdw)
CREATE SERVER file_server FOREIGN DATA WRAPPER file_fdw;

-- CSV file foreign table
CREATE FOREIGN TABLE csv_products (
    id      INT,
    name    TEXT,
    category TEXT,
    price   NUMERIC
)
SERVER file_server

```

```

OPTIONS (
    filename    '/var/lib/postgresql/data/imports/products.csv',
    format      'csv',
    header      'true',
    delimiter    ',',
    null        ''
);

-- Query the CSV
SELECT * FROM csv_products WHERE category = 'electronics' LIMIT 5;
SELECT category, AVG(price) FROM csv_products GROUP BY category;

-- Import CSV data into a real table
INSERT INTO products (name, category, price)
SELECT name, category, price FROM csv_products;

-- Log file analysis (custom log format)
CREATE FOREIGN TABLE pg_log_files (
    log_time      TEXT,
    user_name     TEXT,
    database_name TEXT,
    process_id    INT,
    message       TEXT
)
SERVER file_server
OPTIONS (
    filename '/var/log/postgresql/postgresql-2024-01-15_000000.log',
    format   'csv',
    header   'false'
);

-- Find errors in log
SELECT log_time, message
FROM pg_log_files
WHERE message ILIKE '%ERROR%'
LIMIT 20;

-- TSV (tab-separated) file
CREATE FOREIGN TABLE tsv_data (col1 TEXT, col2 INT, col3 FLOAT)
SERVER file_server
OPTIONS (filename '/tmp/data.tsv', format 'text', delimiter E'\t');

```

Other Common FDWs

mysql_fdw

```

CREATE EXTENSION IF NOT EXISTS mysql_fdw;

CREATE SERVER mysql_server
    FOREIGN DATA WRAPPER mysql_fdw

```

```

OPTIONS (host '10.0.1.60', port '3306');

CREATE USER MAPPING FOR CURRENT_USER
SERVER mysql_server
OPTIONS (username 'mysql_reader', password 'pass');

CREATE FOREIGN TABLE mysql_legacy_data (
  id INT,
  name VARCHAR(100),
  data TEXT
)
SERVER mysql_server
OPTIONS (dbname 'legacy_db', table_name 'customers');

SELECT * FROM mysql_legacy_data LIMIT 10;

```

redis_fdw

```

CREATE EXTENSION IF NOT EXISTS redis_fdw;

CREATE SERVER redis_server
FOREIGN DATA WRAPPER redis_fdw
OPTIONS (address '127.0.0.1', port '6379');

CREATE USER MAPPING FOR PUBLIC
SERVER redis_server OPTIONS (password '');

CREATE FOREIGN TABLE redis_sessions (
  key TEXT,
  val TEXT,
  expire INT
)
SERVER redis_server
OPTIONS (database '0', tabletype 'hash');

```

S3 / Parquet (aws_s3 / parquet_fdw)

```

-- parquet_fdw for reading Parquet files from S3
CREATE EXTENSION IF NOT EXISTS parquet_fdw;

CREATE SERVER s3_server FOREIGN DATA WRAPPER parquet_fdw;

CREATE FOREIGN TABLE s3_events PARTITION OF events_partitioned
FOR VALUES FROM ('2023-01-01') TO ('2024-01-01')
SERVER s3_server
OPTIONS (
  filename 's3://my-bucket/events/2023/*.parquet',
  sorted 'created_at'
);

```

Cross-Database Queries

Within the same PostgreSQL instance, databases are isolated — `postgres_fdw` is needed for cross-database queries.

```
-- Same host, different database
CREATE SERVER local_analytics_server
  FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (
    host 'localhost', -- or use Unix socket
    dbname 'analytics_db',
    port '5432'
  );

CREATE USER MAPPING FOR app_user
  SERVER local_analytics_server
  OPTIONS (user 'analytics_reader', password 'pass');

CREATE FOREIGN TABLE remote_reports (
  report_id INT,
  report_name TEXT,
  data JSONB,
  generated_at TIMESTAMPTZ
)
  SERVER local_analytics_server
  OPTIONS (schema_name 'public', table_name 'reports');

-- Now query across databases from application's main DB
SELECT r.report_name, u.email
FROM remote_reports r
JOIN users u ON u.id = (r.data ->> 'user_id')::int
WHERE r.generated_at > now() - interval '1 day';
```

ETL pipeline with FDW

```
-- Nightly ETL: pull from remote, transform, load locally
CREATE OR REPLACE PROCEDURE sync_orders_from_remote()
LANGUAGE plpgsql AS $$
DECLARE
  v_last_sync TIMESTAMPTZ;
  v_rows_inserted INT;
BEGIN
  -- Find last sync point
  SELECT COALESCE(MAX(created_at), '2020-01-01') INTO v_last_sync
  FROM orders_local;

  -- Insert new remote rows
  INSERT INTO orders_local (id, customer_id, status, total, created_at)
  SELECT id, customer_id, status, total, created_at
```

```

FROM orders_remote
WHERE created_at > v_last_sync
ON CONFLICT (id) DO UPDATE SET
    status = EXCLUDED.status,
    total = EXCLUDED.total;

GET DIAGNOSTICS v_rows_inserted = ROW_COUNT;
RAISE NOTICE 'Synced % rows from remote', v_rows_inserted;
END;
$$;

-- Schedule with pg_cron
SELECT cron.schedule('sync-remote-orders', '0 */1 * * *',
    'CALL sync_orders_from_remote()');
```

Performance Optimization for FDW

```

-- 1. Set fetch_size for batch fetching (default 100, increase for large reads)
ALTER FOREIGN TABLE orders_remote OPTIONS (SET fetch_size '50000');
```

-- 2. Use async execution (PostgreSQL 14+, postgres_fdw 1.1+)

```
ALTER SERVER remote_orders_server OPTIONS (ADD async_capable 'true');
```

-- 3. Create a local materialized view for frequently-joined remote data

```
CREATE MATERIALIZED VIEW orders_remote_cache AS
    SELECT * FROM orders_remote WHERE created_at > now() - interval '90 days';

CREATE UNIQUE INDEX ON orders_remote_cache (id);
```

-- Refresh nightly

```
SELECT cron.schedule('refresh-remote-cache', '0 2 * * *',
    'REFRESH MATERIALIZED VIEW CONCURRENTLY orders_remote_cache');
```

-- 4. Pushdown optimization hints

-- Use exact type matches to help the planner push conditions down

-- BAD: type mismatch prevents pushdown

```
SELECT * FROM orders_remote WHERE total > 100; -- 100 is int, total is numeric
```

-- GOOD: explicit cast

```
SELECT * FROM orders_remote WHERE total > 100::numeric;
```

-- 5. Check remote server statistics

-- On the local server, the remote table has no local statistics.

-- Import them manually:

```
ANALYZE orders_remote; -- This queries remote for statistics
```

-- 6. Row count estimation

```
ALTER FOREIGN TABLE orders_remote OPTIONS (SET use_remote_estimate 'true');
```

-- Causes EXPLAIN to query the remote server for cardinality estimates (slower planning, better plans)

Security Considerations

```
-- 1. Use read-only user on remote side
-- Remote server:
CREATE ROLE fdw_reader LOGIN PASSWORD 'strong_password';
GRANT CONNECT ON DATABASE orders_db TO fdw_reader;
GRANT USAGE ON SCHEMA public TO fdw_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO fdw_reader;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO fdw_reader;

-- 2. Use SSL for connections
CREATE SERVER secure_remote
    FOREIGN DATA WRAPPER postgres_fdw
    OPTIONS (host 'remote-host', dbname 'db', sslmode 'verify-full',
            sslcert '/etc/ssl/client.crt', sslkey '/etc/ssl/client.key');

-- 3. Restrict who can use the user mapping
-- User mappings are per-user – don't create a PUBLIC mapping
-- BAD:
CREATE USER MAPPING FOR PUBLIC SERVER remote_server OPTIONS (...);
-- GOOD:
CREATE USER MAPPING FOR specific_user SERVER remote_server OPTIONS (...);

-- 4. Check who has access
SELECT
    srvname,
    umuser::regrole AS local_user,
    umoptions
FROM pg_user_mappings;

-- 5. Encrypt passwords at rest
-- Passwords in pg_user_mappings are visible to superusers
-- Use vault integration (HashiCorp Vault, AWS Secrets Manager) for production
```

Production Monitoring Queries

```
-- List all foreign servers
SELECT
    s.srvname AS server_name,
    w.fdwname AS fdw_type,
    s.srvoptions AS options,
    r.rolname AS owner
FROM pg_foreign_server s
JOIN pg_foreign_data_wrapper w ON w.oid = s.srvfdw
JOIN pg_roles r ON r.oid = s.srvowner
ORDER BY s.srvname;

-- List all foreign tables with their servers
```

```

SELECT
    ft.foreign_table_schema AS schema,
    ft.foreign_table_name AS table_name,
    ft.foreign_server_name AS server,
    fto.option_value AS remote_table
FROM information_schema.foreign_tables ft
LEFT JOIN information_schema.foreign_table_options fto
    ON fto.foreign_table_schema = ft.foreign_table_schema
    AND fto.foreign_table_name = ft.foreign_table_name
    AND fto.option_name = 'table_name'
ORDER BY ft.foreign_server_name, ft.foreign_table_name;

-- Find slow FDW queries
SELECT
    query,
    calls,
    ROUND(mean_exec_time::numeric, 2) AS avg_ms,
    ROUND(total_exec_time::numeric / 1000, 2) AS total_sec
FROM pg_stat_statements
WHERE query ILIKE '%remote%' -- adjust to your foreign table names
    OR query ILIKE '%fdw%'
ORDER BY mean_exec_time DESC
LIMIT 10;

-- Check active FDW connections (postgres_fdw opens connections to remote)
SELECT
    pid,
    username,
    application_name,
    client_addr,
    state,
    query
FROM pg_stat_activity
WHERE application_name LIKE '%fdw%'
    OR client_addr IS NOT NULL;

-- Foreign table column definitions
SELECT
    table_schema,
    table_name,
    column_name,
    data_type,
    is_nullable
FROM information_schema.columns
WHERE table_name IN (
    SELECT foreign_table_name FROM information_schema.foreign_tables
)
ORDER BY table_name, ordinal_position;

```

Common Mistakes

1. **Not enabling pushdown for aggregates** — By default, `postgres_fdw` doesn't push down aggregates. Set `use_remote_estimate = true` and ensure PostgreSQL version supports aggregate pushdown (9.6+).
 2. **Joining remote tables without local copies** — A complex join between two remote tables fetches all rows from both and joins locally. Cache frequently-joined remote data as materialized views.
 3. **Missing type alignment** — If local column type is `INT` and remote is `BIGINT`, type mismatches can prevent WHERE clause pushdown. Match types exactly.
 4. **Not setting `fetch_size`** — Default `fetch_size` is 100 rows per network round-trip. For large scans, this causes thousands of round-trips. Set `fetch_size = 10000` or higher.
 5. **Using `file_fdw` for production data ingestion** — `file_fdw` files must be on the PostgreSQL server filesystem and readable by the postgres user. It's not suitable for high-throughput ingestion; use COPY instead.
 6. **Forgetting that FDW tables don't support transactions cleanly** — `postgres_fdw` supports 2-phase commit for write transactions (need `two_phase = on` option), but most other FDWs don't. Writes to foreign tables in a transaction that rolls back may not roll back the remote writes.
-

Best Practices

1. **Default to read-only** — FDW is primarily for reading. Write-capable FDWs introduce distributed transaction complexity.
 2. **Cache hot remote data** — Use materialized views for frequently-accessed remote data.
 3. **Use `IMPORT FOREIGN SCHEMA`** — Easier than manual `CREATE FOREIGN TABLE` and stays in sync with remote schema.
 4. **Test pushdown with `EXPLAIN VERBOSE`** — Always verify that WHERE/aggregate pushdown is working.
 5. **Set `application_name`** — Makes it easy to identify FDW connections on the remote server.
-

Interview Questions & Answers

Q1: What is the SQL/MED standard and how does PostgreSQL implement it?

A: SQL/MED (Management of External Data) is an ISO SQL standard extension that defines how relational databases can access external data sources using the same SQL interface. PostgreSQL implements it through Foreign Data Wrappers (FDW): a foreign server defines the connection, a user mapping provides credentials, and foreign tables provide a local interface to remote data. The FDW plugin translates SQL operations into the native protocol of the external source.

Q2: What is "pushdown" in the context of `postgres_fdw`?

A: Pushdown is the optimization where the FDW sends query conditions (WHERE clauses, JOINS, aggregates) to the remote server for execution, rather than fetching all rows and filtering/aggregating locally. For a large remote table with `WHERE status = 'active'`, pushdown means only matching rows travel the network. `EXPLAIN VERBOSE` shows the "Remote SQL" that gets executed on the remote server.

Q3: What are the limitations of foreign tables compared to regular tables?

A: Foreign tables: (1) don't support local indexes, (2) have inaccurate row count estimates unless `ANALYZE` is run or `use_remote_estimate = true`, (3) don't support local constraints (CHECK, FK), (4) write support is

FDW-dependent and often lacks full transaction support, (5) are slower than local tables due to network overhead, (6) don't support `VACUUM` or `ANALYZE` in the same way.

Q4: When would you use `file_fdw` ?

A: `file_fdw` is useful for: (1) bulk-importing CSV/TSV files using SQL queries, (2) ad-hoc analysis of log files using SQL, (3) loading configuration files that change infrequently. It's not suitable for production data pipelines because files must be on the PostgreSQL server filesystem, it's read-only, and `COPY` is faster for bulk loading.

Q5: How do you cross-database query within the same PostgreSQL instance?

A: PostgreSQL databases within an instance are isolated — you cannot directly `SELECT` from another database. Use `postgres_fdw` with `host=localhost` and the target `dbname` . This opens a new libpq connection to the same server but a different database. Alternatively, use schemas within a single database (schemas don't have this limitation).

Q6: What is `use_remote_estimate` and what are its trade-offs?

A: When `use_remote_estimate = true` , `postgres_fdw` queries the remote server for row count estimates during query planning (via `EXPLAIN` on the remote query). This gives the local planner accurate cardinality estimates for better plan choices. The trade-off is that every planning phase makes an extra round-trip to the remote server, slightly increasing query startup latency. Use it for complex queries where accurate cardinality matters; skip it for simple OLTP queries.

Exercises and Solutions

Exercise 1

Set up `file_fdw` to query a CSV file containing product data. Import the data into a real table. Show row counts before and after.

Solution:

```
-- First, create a test CSV file (as postgres OS user):
-- echo "id,name,price" > /tmp/products.csv
-- echo "1,Widget A,9.99" >> /tmp/products.csv
-- echo "2,Widget B,19.99" >> /tmp/products.csv
-- echo "3,Widget C,29.99" >> /tmp/products.csv

CREATE EXTENSION IF NOT EXISTS file_fdw;
CREATE SERVER file_srv FOREIGN DATA WRAPPER file_fdw;

CREATE FOREIGN TABLE csv_import (
    id INT,
    name TEXT,
    price NUMERIC
)
SERVER file_srv
OPTIONS (filename '/tmp/products.csv', format 'csv', header 'true');

SELECT COUNT(*) FROM csv_import; -- 3
```

```
CREATE TABLE products_imported AS SELECT * FROM csv_import;
SELECT COUNT(*) FROM products_imported; -- 3
```

Exercise 2

Write a query using `IMPORT FOREIGN SCHEMA` and then verify pushdown works for a filtered query.

Solution:

```
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

CREATE SERVER demo_remote FOREIGN DATA WRAPPER postgres_fdw
OPTIONS (host 'localhost', dbname 'postgres', port '5432');

CREATE USER MAPPING FOR CURRENT_USER SERVER demo_remote
OPTIONS (user 'postgres', password '');

CREATE SCHEMA IF NOT EXISTS imported_remote;

IMPORT FOREIGN SCHEMA public
LIMIT TO (pg_stat_statements) -- import pg_stat_statements as foreign table
FROM SERVER demo_remote
INTO imported_remote;

-- Verify pushdown
EXPLAIN VERBOSE
SELECT * FROM imported_remote.pg_stat_statements
WHERE calls > 100;
-- Look for "Remote SQL: SELECT ... WHERE (calls > 100)"
```

Cross-References

- **06_logical_decoding.md** — Alternative approach to cross-system data sync
- **04_materialized_views.md** — Caching remote FDW data locally
- **14_security** — Credential management for FDW user mappings
- **05_extensions_overview.md** — Extension management
- **PostgreSQL Docs** — <https://www.postgresql.org/docs/current/postgres-fdw.html>